```
import pandas as pd
```

```
df = pd.read_csv('emails.csv')
```

```
df.shape
```

```
(5172, 3002)
```

```
df.head()
```

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	valued	lay	infrastructure	military
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	0	0	0	0
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	0	0	0	0
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	0	0	0	0
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	0	0	0	0
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	0	0	0	0

5 rows × 3002 columns

```
# Input Data
x= df.drop(['Email No.', 'Prediction'], axis=1)

# Output Data
y = df['Prediction']
```

```
# Checking shape of x
x.shape

# Checking datatypes in x
x.dtypes

# All are int64 so no need to clean the data
```

	0
the	int64
to	int64
ect	int64
and	int64
for	int64
...	...
infrastructure	int64
military	int64
allowing	int64
ff	int64
dry	int64

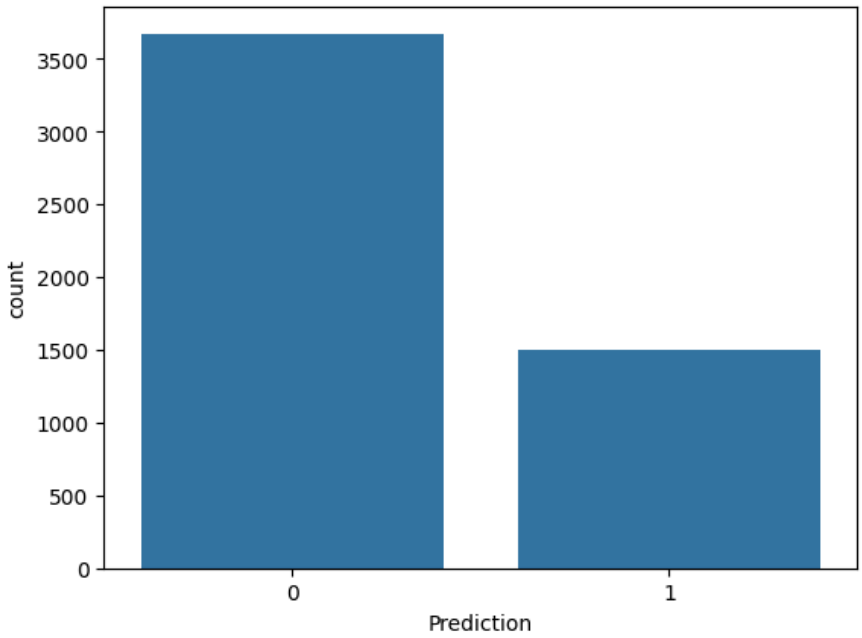
3000 rows × 1 columns

dtype: object

```
import seaborn as sns

# Plotting x and y
sns.countplot(x = y)
```

```
<Axes: xlabel='Prediction', ylabel='count'>
```




```
# Checking the count of 0 and 1 in y
y.value_counts()
```

Prediction	count
0	3672
1	1500

dtype: int64

```
# Feature scaling
# Mini max scaling of x

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()

x_scaled = scaler.fit_transform(x)
```

```
# Checking values in the x_scaled
x_scaled
```

```
array([[0.        , 0.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.03809524, 0.09848485, 0.06705539, ..., 0.        , 0.00877193,
        0.        ],
       [0.        , 0.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       ...,
       [0.        , 0.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.00952381, 0.0530303 , 0.        , ..., 0.        , 0.00877193,
        0.        ],
       [0.1047619 , 0.18181818, 0.01166181, ..., 0.        , 0.        ,
        0.        ]])
```

```
# Cross Validation
# Spliting the data into training and testing (75% & 25%)
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x_scaled, y, test_size=0.25, random_state=0)
```

```
# Checking if the split has done or not
x_scaled.shape
```

```
(5172, 3000)
```

```
x_train.shape
```

```
(3879, 3000)
```

```
x_test.shape
```

```
(1293, 3000)
```

```
# KNN Algorithm
# Importing the libraries
from sklearn.neighbors import KNeighborsClassifier

# creating object (value of k = 5)
knn = KNeighborsClassifier(n_neighbors=5)
```

```
# Train the algorithm
knn.fit(x_train, y_train)
```

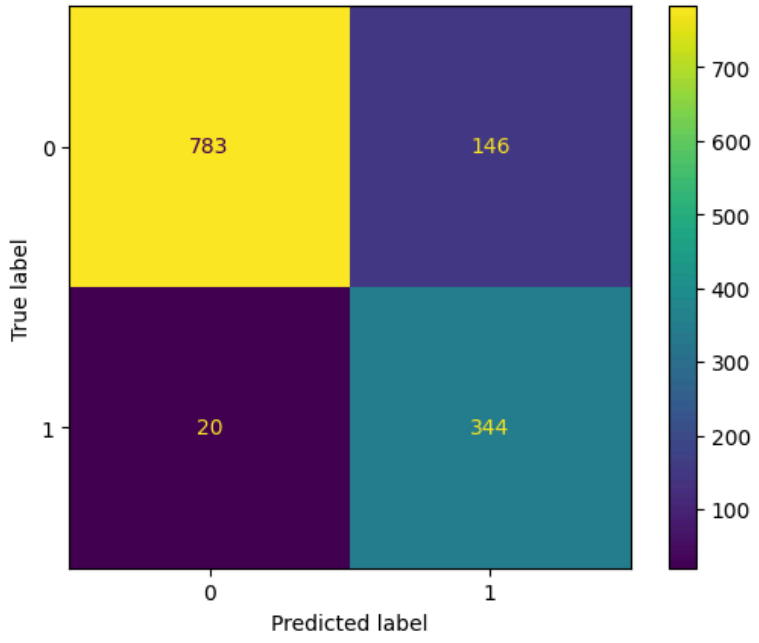
```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
# Predicting on the test data
y_pred = knn.predict(x_test)
```

```
# Need evaluation matrix for this (confusion matrix)
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score
from sklearn.metrics import classification_report
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7dcffeb23f50>
```




```
y_test.value_counts()
```

Prediction	count
0	929
1	364

dtype: int64

```
# Accuracy Score

accuracy_score(y_test, y_pred)
```

```
0.871616395978345
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.98      0.84      0.90       929
           1       0.70      0.95      0.81       364

    accuracy                           0.87      1293
   macro avg       0.84      0.89      0.85      1293
weighted avg       0.90      0.87      0.88      1293
```

```
# For finding the best value of k
# calculating the array of Error rate to check for which value of k the error rate is minimum
import numpy as np

Error_rate = []
for i in range(1, 41):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(x_train, y_train)
    pred_i = knn.predict(x_test)
    Error_rate.append(np.mean(pred_i != y_test))

Error_rate
```

```
# From above we can see that for k = 1 the accuracy can be increased
```